



CARRERA DE ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL (CEIA)

ALGORITMOS EVOLUTIVOS I

TRABAJO FINAL

El problema de la mochila en alpinismo

ALUMNO: Mauro Fabian Barquinero

AÑO: 2025

1. Resumen

Se aborda el armado de una mochila de alpinismo como un problema de la mochila, donde cada elemento del catálogo posee peso (kg) y utilidad. El objetivo es seleccionar el subconjunto de ítems que maximiza la utilidad total respetando una capacidad de carga. Para resolverlo se implementó un Algoritmo Genético (GA) con codificación binaria, selección por ruleta, cruce de un punto, mutación por bit y elitismo. A fin de reflejar criterios prácticos del dominio (seguridad y coherencia técnica), se incorporan restricciones suaves en la función de aptitud: bonificaciones por ítems críticos, penalizaciones por faltantes y por sobrepeso.

2. Explicación del problema y abordaje de la solución

2.1. ¿En qué consiste el problema?

El objetivo es decidir qué elementos llevar en la mochila para una salida de alpinismo, de modo que el conjunto sea seguro, útil y factible de transportar. Se dispone de un catálogo de ítems que cada uno tiene un peso (kg) y una utilidad. La mochila admite una capacidad máxima (W). La decisión es binaria: incluir o no cada ítem. El reto práctico es que, más allá de “llenar hasta el peso”, debemos lograr un equilibrio razonable entre hidratación, alimentación, primeros auxilios, abrigo y equipo técnico.

2.2. Enfoque elegido

Se optó por un Algoritmo Genético (GA) con representación binaria (un bit por ítem). El GA mantiene una población de soluciones y, generación tras generación, evalúa, selecciona, recombina y muta candidatos, preservando siempre el mejor.

Para guiar al GA sin imponer listas rígidas de “obligatorios”, se definió una función de aptitud con restricciones suaves:

- Utilidad base: suma de utilidades de los ítems seleccionados.
- Bonificaciones: pequeños “empujes” positivos para elementos críticos (agua, comida, botiquín, navegación) y para el equipo técnico (cuerda, arnés, casco).
- Penalizaciones por faltantes: descuentos si esos críticos no están presentes (no prohíbe, pero desalienta).
- Dependencias suaves: restan puntaje si se viola una coherencia.
- Sobrepeso: castigo proporcional si el peso total supera la capacidad (W).

Este diseño permite que el algoritmo descubra configuraciones válidas dentro del límite de peso, privilegiando seguridad y coherencia técnica.

2.3 Implementación en términos prácticos

- Catálogo: lista (nombre, peso, utilidad); se partió de una base y se amplió, ajustando algunos pesos mínimos en accesorios livianos para que no dominen la solución por simple acumulación.
- Capacidad: se fijó una capacidad moderada ($W \approx 9.2$ kg), representativa de una mochila con equipo técnico.

- GA: población de tamaño medio, cruce de un punto, mutación por bit y elitismo. Los parámetros se calibraron empíricamente observando la curva de convergencia (mejor y promedio).

2.4 Inconvenientes y cómo se resolvieron

(i) Exceso de accesorios “livianos”

- *Síntoma*: las primeras ejecuciones incluían muchos ítems de bajo impacto, porque su peso reducía poco la capacidad.
- *Corrección*: se redondearon al alza algunos pesos mínimos y liberó espacio para esenciales.

(ii) Ausencia de agua y comida en soluciones tempranas

- *Síntoma*: el GA prefería llenar el peso con varios accesorios y “pagar” el penal por faltar agua/comida.
- *Corrección*: se reforzaron las bonificaciones y, sobre todo, las penalizaciones por ausencia de agua y comida. Con ello el algoritmo pasó a priorizarlas de forma consistente.

(iii) Inclusión inestable de la cuerda (equipo técnico)

- *Síntoma*: dependiendo del azar, la cuerda no aparecía en la mejor solución, a pesar del contexto.
- *Corrección*: se elevaron suavemente el bono y el penal por ausencia de la cuerda y se activaron dependencias mínimas. Dando el resultado que la cuerda entra desplazando extras livianos cuando corresponde.

(iv) Tamaño del set demasiado grande

- *Síntoma*: el número de ítems era alto, aunque el peso total fuera factible.
- *Corrección*: se redujo ligeramente la capacidad (W) (de 9.9 a 9.2 kg), lo que compactó la lista a 20 ítems manteniendo seguridad y funcionalidad.

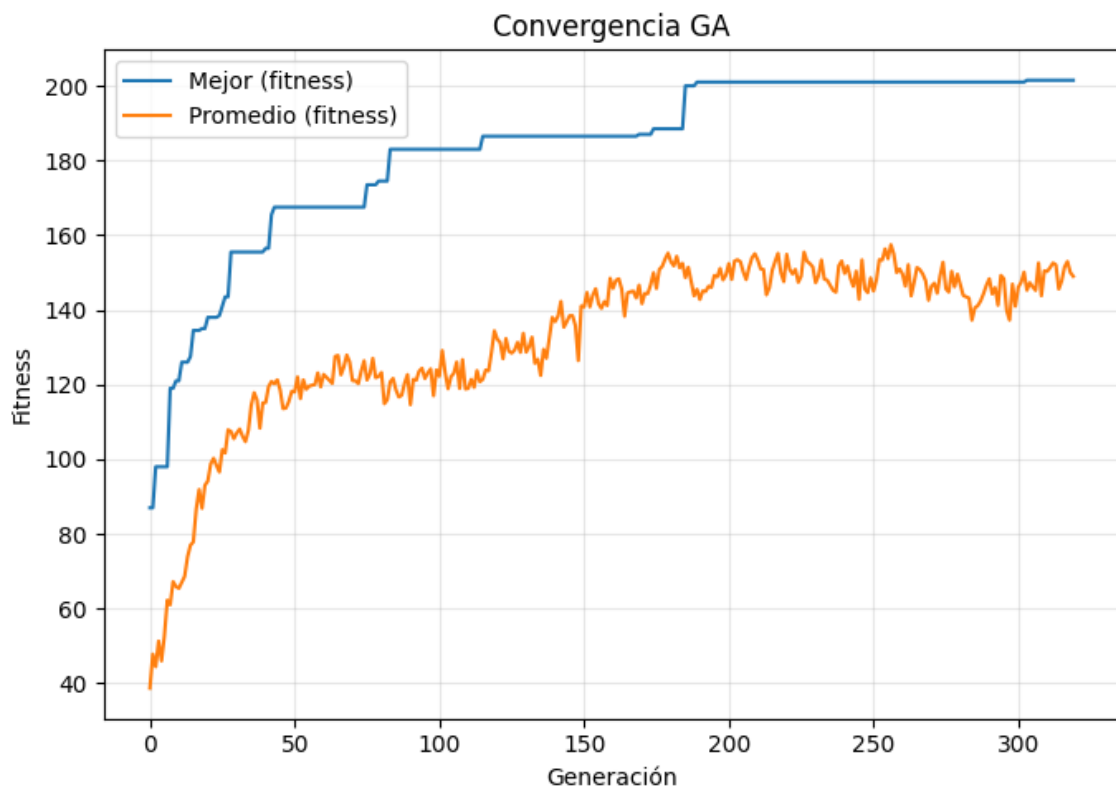
2.5 Resultado final

Tras los ajustes, el GA converge de forma estable a mochilas compactas y realistas: incluyen agua, comida, botiquín, navegación, linterna, capas clave (capa térmica y campera impermeable) y equipo técnico consistente (cuerda, arnés, casco y asegurador), respetando la capacidad. La convergencia por escalones y la estabilización del mejor fitness indican una optimización efectiva respecto del modelo planteado.

2.6 Observaciones y alcance

- El enfoque de restricciones suaves evita imponer listas fijas y, en su lugar, incentiva los comportamientos deseados y desalienta las incoherencias, a la vez que mantiene el carácter exploratorio del GA.
- Las utilidades son heurísticas (criterio experto); aun así, el método permite ajustarlas y recalibrar bonificaciones/penales según terreno, clima o política de la empresa.
- El marco es extensible (por ejemplo: añadir volumen como segunda restricción o comparar con otros metaheurísticos), pero ya cumple su propósito: una solución factible, segura y argumentada para el armado de la mochila.

3. Gráfico de convergencia



- Ascenso rápido inicial (gen 0–40): el “mejor fitness” (azul) crece en forma pronunciada. Es la fase de exploración, la población descubre combinaciones de alto impacto (agua, comida, botiquín, navegación, frontal) y descarta configuraciones con sobrepeso o incoherencias.
- Escalones sucesivos (gen 40–200): la curva azul progresa a saltos. Cada “escalón” refleja un hallazgo discreto relevante (por ej., reemplazar varios accesorios de bajo valor por una capa clave o por equipo técnico consistente como cuerda y arnés/casco/ATC).
- Meseta final (gen 200→fin): la azul se estabiliza en torno al valor máximo observado. Indica convergencia: con los operadores y parámetros utilizados, la población ya no encuentra mejoras significativas.
- Promedio de fitness (naranja): sube de manera sostenida y luego oscila en una franja estable, por debajo del mejor. Esto muestra que:
 - se mantiene diversidad (no todos los individuos son iguales al mejor).

- la población se concentra gradualmente alrededor de buenas soluciones sin colapsar en un único punto demasiado pronto (no hay estancamiento prematuro).

Resumen: En términos prácticos, el algoritmo alcanzó un óptimo local robusto para la función objetivo que combina la utilidad con bonificaciones y penalizaciones por sobrepeso, y generó una mochila factible, compacta y técnicamente coherente.

4. Explicaciones de lo más relevante del código fuente

4.1 Estructura de datos

- Catálogo de ítems: lista de tuplas (nombre, peso_kg, utilidad). Es fácil de leer y editar; si se cambia un peso o una utilidad, el GA lo incorpora sin tocar el resto del código.
- Parámetros del modelo:
 - W (capacidad de la mochila) y LAMBDA_W (penal por sobrepeso).
 - Diccionarios BONUS y PENAL_MISS que asignan bonificaciones y penalizaciones a nombres de ítems.
 - Lista DEPENDENCIES con reglas suaves.

4.2 Funciones clave

- `chrom_weight_value(ch)`: recibe un cromosoma binario (lista de 0/1) y devuelve peso total y utilidad base de los ítems seleccionados.
- `fitness(ch)`: define qué es una buena mochila. Parte de la utilidad base y le suma:
 - Bonos por críticos presentes (BONUS).
 - Penalidades por críticos ausentes (PENAL_MISS).
 - Penalidades por dependencias incumplidas (DEPENDENCIES, p. ej. ATC sin cuerda).
 - Penalidades por sobrepeso si el total excede W (proporcional al exceso, controlado por LAMBDA_W).

Esta función encapsula todo el conocimiento de dominio sin bloquear opciones: orienta al GA a “descubrir” combinaciones seguras y coherentes sin imponer obligatorios duros.

4.3 Operadores evolutivos

- `rand_chrom()` y `gen_pop(n)`: crean soluciones aleatorias y la población inicial.
- `select_parents(pop)`: ruleta proporcional al fitness con salvaguarda (si la suma de aptitudes positivas es muy baja, selecciona al azar). Evita estancamientos y mantiene diversidad.
- `crossover(p1, p2)`: cruce de un punto. Recombina subconjuntos de ítems de manera simple y eficaz para mochila 0/1.
- `mutate(ch)`: invierte un bit aleatorio. La probabilidad PMUT relativamente alta favorece explorar y escapar de óptimos locales.
- `best_of(pop)`: devuelve el mejor individuo según fitness. Se usa para elitismo (preservar el mejor de cada generación) y para reportes.

4.4 Bucle evolutivo

1. Inicialización: `pop = gen_pop(POP)`.
2. Evolución durante GEN generaciones:
 - Evaluar fitness de la población.
 - Elitismo: guardar el mejor.
 - Repetir selección → cruce → mutación hasta completar la nueva población.
 - Registrar mejor y promedio por generación para graficar convergencia.
3. Salida: mejor cromosoma, su peso, utilidad y listado de ítems seleccionados.

El código registra una curva de convergencia (mejor/promedio), que sirve para evidenciar aprendizaje y estabilidad.

4.5 Hiperparámetros y reproducibilidad

- POP, GEN, PMUT, ELITISM controlan diversidad, explotación y tiempo de cómputo. Se eligieron valores moderados que muestran convergencia estable.
- La semilla (`random.seed(...)`) permite reproducir corridas.